# Labo 3: Regex Engine

206.3 Formal and Natural Languages (FNL)

## Introduction, Objectifs

Ce labo vous permettra d'explorer la correspondance entre expressions régulières (**regex**) et automates finis non déterministes (**AFN**), en implémentant l'algorithme de Thompson. Les objectifs sont de :

- comprendre comment décomposer une regex en unités élémentaires et transformer chacune de ces unités en petits automates à l'aide d'opérations (symboles, concaténation, union, étoile).
- créer en python un **moteur de regex** qui pourra tester si une regex accepte une chaîne de caractères donnée.

## Partie 1: Squelette de code pour l'AFN

Voici un squelette minimal pour représenter un AFN (à copier dans un fichier Python nommé `nfa.py`):

```
class State:
    '''Un état d un AFN avec un identifiant séquentiel lisible.'''
    _next_id = 0  # attribut de classe servant de compteur d'ID

    def __init__(self):
        # transitions : dictionnaire { symbole (None pour epsilon): [états
            cibles] }
        self.transitions = {}
        # affectation d'un identifiant unique et lisible à chaque état créé
        self.id = State._next_id
        State._next_id += 1

    def add_transition(self, symbol, state):
        '''ajoute une transition vers state avec le symbole (None pour epsilon)
            .'''
        if symbol in self.transitions:
            self.transitions[symbol].append(state)
        else:
            self.transitions[symbol] = [state]

    def __str__(self):
        lst = []
        for symbole, states in self.transitions.items():
            s_symbole = symbole if symbole is not None else 'ε'
            ids = [s.id for s in states]
            lst.append(f"{s_symbole} -> {ids}")
        return f"State({self.id}): " + ", ".join(lst)


class NFA:
    '''Un AFN qui posséde un état initial et un état final.'''
    def __init__(self, start, finish):
```

*Listing continues on next page...*

*...continuing from previous page*

```
        self.start = start
        self.finish = finish

    def display(self):
        '''affichage simple de l AFN par parcours en profondeur.'''
        visited = set()

        def dfs(state):
            if state in visited:
                return
            visited.add(state)
            print(state)
            for nexts in state.transitions.values():
                for s in nexts:
                    dfs(s)  # récursif
        dfs(self.start)
```

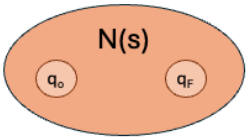
Ce squelette vous permet de créer un AFN, par exemple:

```
# Création d'un AFN simple pour le symbole 'a'
start = State()
finish = State()
start.add_transition('a', finish)
nfa_symbol_a = NFA(start, finish)
nfa_symbol_a.display()
```

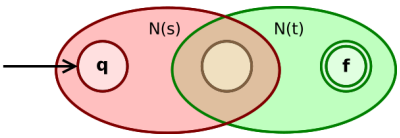
### Partie 2: Algorithme de Thompson

L'algorithme de Thompson permet de construire un AFN pour une regex donnée en *combinant de petits automates*. Voici les 4 opérations de base qui combinent des AFN pour former un AFN plus complexe:

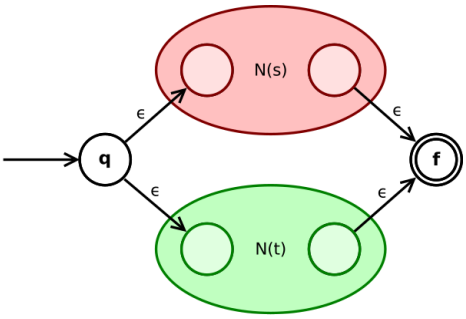
a) Pour un **symbole** (par exemple `"a"`, ou `"b"`, etc.), on crée deux états reliés par une transition labellisée. Pour réaliser cette opération, vous créerez la fonction `symbol(sym)` qui créera l'AFN d'un symbole simple.



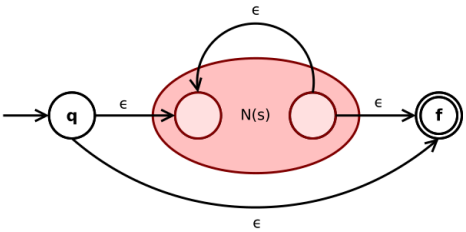

b) Pour la **concaténation** de deux automates, on relie l'état final du premier à l'état initial du second via une transition epsilon. La concaténation sera *implicite*, par exemple "ab" signifie a suivi de b. Pour cela, vous créerez la fonction `concat(nfa1, nfa2)-> NFA` qui concatènera deux AFN.




c) Pour l'**union** (opérateur `|`), on crée un nouvel état initial pointant en epsilon vers chacun des sous-automates et un nouvel état final accessible en epsilon depuis les états finaux. Pour cela, vous créerez la fonction `union(nfa1, nfa2)-> NFA` qui créera l'union de deux AFN.




d) Pour l'**étoile** (opérateur `*`), on crée un nouvel état initial/final connecté en epsilon à l'automate, qui peut être parcouru plusieurs fois. Pour cela, vous créerez la fonction `star(nfa)-> NFA` qui appliquera l'étoile de Kleene sur un AFN.




**Implémentation**

Implémentez ces 4 fonctions de l'algorithme de Thompson en suivant les spécifications ci-dessous. Relisez bien les descriptions/images ci-dessus pour vous aider à comprendre comment coder vos 4 fonctions. Aidez-vous également des tests unitaires fournis dans le fichier `test_nfa.py` pour vérifier votre implémentation.

a) Fonction `symbol(sym)-> NFA` :

- Créez deux états (start et finish) avec l'objet `State`.
- Ajoutez une transition entre start et finish, avec le symbole `sym` donné en paramètre.
- Retournez un nouvel objet NFA qui commence par `start` et finit par `finish`.

b) Fonction `concat(nfa1, nfa2)-> NFA` :

- Ajoutez une transition epsilon (càd sans symbole, et donc représentée par `None`) depuis l'état final de `nfa1` vers l'état initial de `nfa2`.
- Retournez un nouvel AFN qui commence par `nfa1.start` et finit par `nfa2.finish`.

c) Fonction `union(nfa1, nfa2)-> NFA` :

- Créez un nouvel état `new_start` et un état `new_finish`.
- Depuis `new_start`, ajoutez des transitions epsilon vers `nfa1.start` et `nfa2.start`.
- Depuis les états finis de `nfa1` et `nfa2`, ajoutez une transition epsilon vers `new_finish`.
- Retournez le nouvel AFN, qui commence par `new_start` et finit par `new_finish`.

d) Fonction `star(nfa)-> NFA` :

- Créez un nouvel état `new_start` et un nouvel état `new_finish`.
- Depuis `new_ start`, ajoutez une transition epsilon vers `nfa.finish` et une autre vers `nfa.start`.
- Depuis `nfa.finish`, ajoutez une transition epsilon vers `nfa.start` et une vers `new_finish`.

- Cette construction permet d'accepter la chaîne vide.
- Retournez le nouvel AFN, qui commence par `new_start` et finit par `new_finish`.

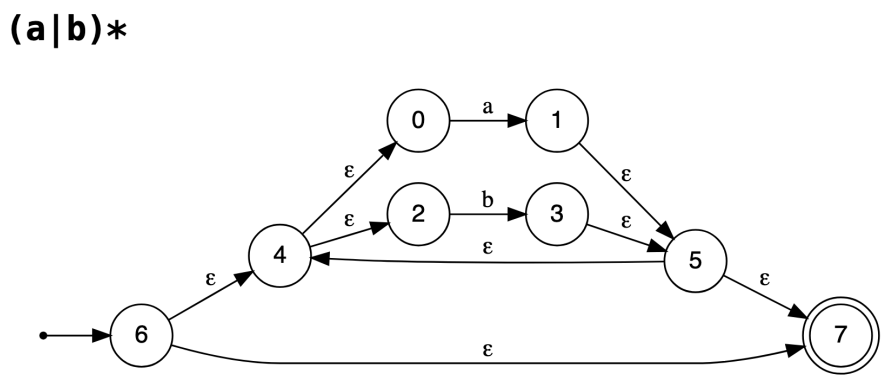
Vous pouvez ensuite utiliser vos fonctions en utilisant le code suivant (à copier à la fin de votre fichier `nfa.py`) :

```
if __name__ == '__main__':
    from utils import export_html

    # Etape 1: construire un AFN pour la regex "a|b"
    nfa_a = symbol('a')
    nfa_b = symbol('b')
    nfa_union = union(nfa_a, nfa_b)
    # Etape 2: construire un AFN pour la regex "(a|b)*"
    nfa_star = star(nfa_union)

    # Afficher l'AFN
    print("AFN pour (a|b)* :")
    nfa_star.display()
    # Exporter l'AFN en HTML pour visualisation graphique
    export_html(nfa_star, nfa_name="(a|b)*", filename="nfa_star.html")
```

Cette petite démo devrait ouvrir votre navigateur web avec cet AFN:



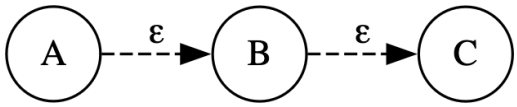

### Partie 3: Epsilon Closure

Notre but ultime dans ce labo est d'avoir un programme capable de **tester si une regex (transformée en AFN) accepte une chaîne de caractères donnée**. Pour cela, nous devons implémenter la *fermeture epsilon*.

### Qu'est-ce que la fermeture epsilon ?

Tout d'abord, comprenons le terme de ***fermeture*** (*closure*) en général: Imaginons un réseau de villes reliées par des routes à sens unique : si depuis la ville A on peut aller directement à la ville B, et depuis B on peut aller directement à C, alors la *fermeture* de `{A}` par rapport à l'opération "voyager par route" serait `{A, B, C}`. Cette fermeture représente l'ensemble complet des destinations accessibles en partant du point initial A et en suivant les chemins disponibles.

La **fermeture epsilon (*ε-closure*)** d'un ensemble des états d'un AFN est l'ensemble de tous les états accessibles à partir de ces états, via uniquement des transitions epsilon (c'est-à-dire *sans* consommer de symbole). Par exemple, si un état A possède une transition epsilon vers l'état B, et B possède à son tour une transition epsilon vers C, alors l'epsilon closure de `{A}` est `{A, B, C}`, c'est-à-dire que l'on peut être à l'état A, passer de A à B et passer de A à B à C sans consommer de symbole.




Cette opération est essentielle dans la simulation d'un AFN :

- Avant de lire le premier symbole d'une chaîne, on considère tous les états accessibles *« gratuitement »* (via des transitions epsilon) depuis l'état initial.
- Après avoir consommé un symbole, on calcule à nouveau la fermeture epsilon des états atteints pour prendre en compte les transitions epsilon qui suivent.

### Implémentation de l'epsilon closure

Vous trouverez dans le fichier `nfa_regex_engine.py` l'implémentation de la fonction `epsilon_closure`, accompagnée de commentaires pour détailler chaque étape de l'algorithme :

1. initialisation d'une pile (stack) avec tous les états de départ.
2. initialisation de `closure` avec l'ensemble initial.
3. Tant que la pile n'est pas vide, on dépile un état.
4. Pour cet état, si une transition epsilon existe (représentée par le symbole `None`), on parcourt chacun des états cibles et, s'ils ne font pas déjà partie de la fermeture, on les ajoute et on les empile.
5. À la fin, closure contient tous les états atteignables via zéro ou plusieurs transitions epsilon.

Lancez le script `nfa_regex_engine.py` et suivez les 2 exemples de code dans la `main` pour être sûr que vous avez bien compris l'algorithme. Il y a 2 exemples de code qui illustrent la façon dont l'epsilon closure fonctionne pour :

- un état sans transition (exemple 1)
- une transition directe (exemple 2)

Ajoutez des exemples supplémentaires pour illustrer les cas suivants (regardez le code pour les exemples):

- une chaîne de transitions (exemple 3)
- des branches (exemple 4)
- un cycle (exemple 5)

Pour chacun des exemples, nous affichons l'ensemble des id des états qui font partie de la fermeture epsilon à partir de l'état de départ. Cette approche aide à comprendre comment et pourquoi la fermeture epsilon permet à un AFN de "se propager" sans consommer de symbole.

### Tester votre implémentation

Voici le moment tant attendu où vous pouvez tester votre moteur de regex! Créez un fichier `regex_engine_demo.py` et créez un AFN pour la regex `(ab|cd)*`. Ensuite, testez avec la fonction `accepts` du fichier `nfa_regex_engine.py` si l'AFN accepte les chaînes suivantes:

```
testcases = [
    'ab',          # True
    'cd',          # True
    'a',           # True
    'b',           # True
    'cg',          # False
    'd',           # True
    'abab',        # True
    'cdcdc',       # True
    'abcd',        # True
    'abe',         # False
    'abhi',        # False
    'cdjk',        # False
    'ababhi',      # False
    'cdjkcdjk'     # False
]
```

## Partie 4: Démonstration de votre moteur de regex

Vous allez maintenant comparer les performances de votre moteur de regex avec celles du module `re` de Python. Voici les étapes, à réaliser dans votre fichier `regex_engine_demo.py`:

1. Création de l'automate :
   - Utilisez vos fonctions `symbol`, `concat`, et `star` pour construire un NFA qui reconnaît l'expression régulière `a(bc)*(de)*f`.
   - Assurez-vous que votre AFN est correctement configuré, checkez-le comme ci-dessus avec la fonction `accepts` de `nfa_regex_engine.py`, par exemple avec `accepts(nfa, "abcde")`.
2. Génération d'un fichier de test :
   - Écrivez une fonction `generate_large_file` qui crée un fichier texte contenant 1 million de lignes. Chaque ligne doit être une chaîne aléatoire composée des lettres 'a' à 'f', avec une longueur aléatoire entre 5 et 15 caractères.
3. Test du moteur :
   - Implémentez une fonction `test_nfa_on_file` qui lit chaque ligne du fichier généré et utilise votre NFA pour vérifier si la ligne correspond à l'expression régulière.
   - Comptez et affichez le nombre de lignes qui correspondent, ainsi que le temps total pris par le NFA pour traiter l'entier du fichier.

4. Comparaison avec le module `re` de Python :
   - Écrivez une fonction `test_re_on_file` qui utilise le module `re` pour effectuer la même vérification sur chaque ligne du fichier.
   - Utilisez `re.fullmatch` pour vous assurer que l'expression régulière correspond à la ligne entière.
   - Comptez et affichez le nombre de lignes qui correspondent, ainsi que le temps total pris par `re` pour traiter le fichier.
5. Analyse des résultats :
   - Comparez les performances du moteur NFA avec celles du module `re`. Discutez des différences de temps d'exécution et de la précision des résultats.

### Autres opérateurs [optionnel]

### Analyseur syntaxique (parser) [avancé et optionnel]

Jusqu'ici, nous devons créer un AFN en python, pièce par pièce, en utilisant les fonctions `symbol`, `concat`, `union` et `star`. Ce qui serait vraiment chouette, c'est si nous pouvions utiliser *directement* une regex (fournie sous forme de chaîne de caractères) pour créer l'AFN. Par exemple, si nous pouvions utiliser une fonction ainsi pour créer l'AFN:

```
nfa = regex_to_nfa("(ab|cd)*")  # crée un AFN pour la regex "(ab|cd)*"
```

Cela s'appelle un **analyseur syntaxique** (*parser*), et c'est ce que je vous propose d'implémenter dans cette dernière partie. Vous aurez l'occasion de développer un parseur plus évolué dans le cours de programmation fonctionnelle.

Vous trouverez dans le fichier `parser.py` un squelette d'analyseur syntaxique:

- La fonction `regex_to_nfa` définit en interne les 4 fonctions de parsing nécessaires et appelle `parse_expr(0)` pour démarrer.
- Chacune des fonctions retourne un tuple `(nfa, nouvelle_position)` qui permet de suivre le parcours de la chaîne.
- Les commentaires exhaustifs décrivent les étapes que vous devez suivre pour implémenter chaque fonction.

Complétez chaque fonction en implémentant le code correspondant à la logique décrite. Cela vous permettra de construire progressivement votre analyseur syntaxique de regex en AFN.

Pour tester votre analyseur syntaxique, vous pouvez utiliser le code suivant:

```
nfa = regex_to_nfa("(ab|cd)*")
nfa.display()
```

#### Sources

https://williamsantos.me/posts/2022/regex-engine-first-principles-golang/#org51aae97